

SERV Core Integration into Chipyard: An Overview

*A focused guide on embedding the ultra-minimal SERV core within the Chipyard framework
using wrappers, tiles, and adapter chains.*

List of Figures

Figure 4-1: Integration Flow6

Figure 4-3: AXI-Tilelink Adapter Chain..... 16

Figure 4-4: Tilelink-AXI Adapter Chain..... 16

Figure 4-5: Filesystem for Integration 17

Table of Contents

Table of Contents	III
Core Integration Overview	6
1.1 Introduction.....	6
1.1.1 Overview of integration steps	6
1.2 BlackBox Wrapping for Core Integration.....	6
1.2.1 Verilog BlackBox Wrapping	6
1.2.2 Scala BlackBox Definition	7
1.2.3 Practical Considerations and Limitations	7
1.3 Defining Core and Tile Parameters.....	8
1.3.1 Crafting CoreParams and TileParams case classes	8
1.3.2 InstantiableTileParams.....	9
1.3.3 Documentation and handling of “Rocket-specific” parameters.....	9
1.4 The Tile Class	10
1.4.1 Custom Class	10
1.4.2 Connecting TileLink master/slave nodes to diplomatic fabric	11
1.4.3 Handling device description and resource registration.....	11
1.5 Tile Module Implementation	12
1.5.1 TileModuleImp	12
1.5.2 Connecting core blackbox to TileLink or AXI interfaces.....	12
1.5.3 Interrupt Handling.....	13
1.6 Configuring Core in Chipyard Build Flow	14
1.6.1 Writing Config Fragments to modify TilesLocated.	14
1.6.2 CanAttachTile and TileAttachParams.....	14
1.7 Protocol Bridging (if Non–TileLink Core)	15

1.7.1	Identifying interface mismatches	15
1.7.2	Inserting diplomatic adapters	15
1.8	Integration Validation.....	17
1.9	Summary	18

It is hereby intended to provide a tutorial-style guide on integrating the SERV core into a complete RISC-V SoC.

Core Integration Overview

1.1 Introduction

The SERV core, a bit-serial, minimal RISC-V CPU, is integrated into the Chipyard framework. SERV implements the RV32I ISA with exceptional area efficiency, making it ideal for systems with limited resources and ultra-low power consumption. The architectural distinctions between the Rocket and BOOM cores, significant integration difficulties, and verification techniques used to ensure correct functionality are also highlighted.

1.1.1 Overview of integration steps

The step-by-step roadmap of the core integration is as under:

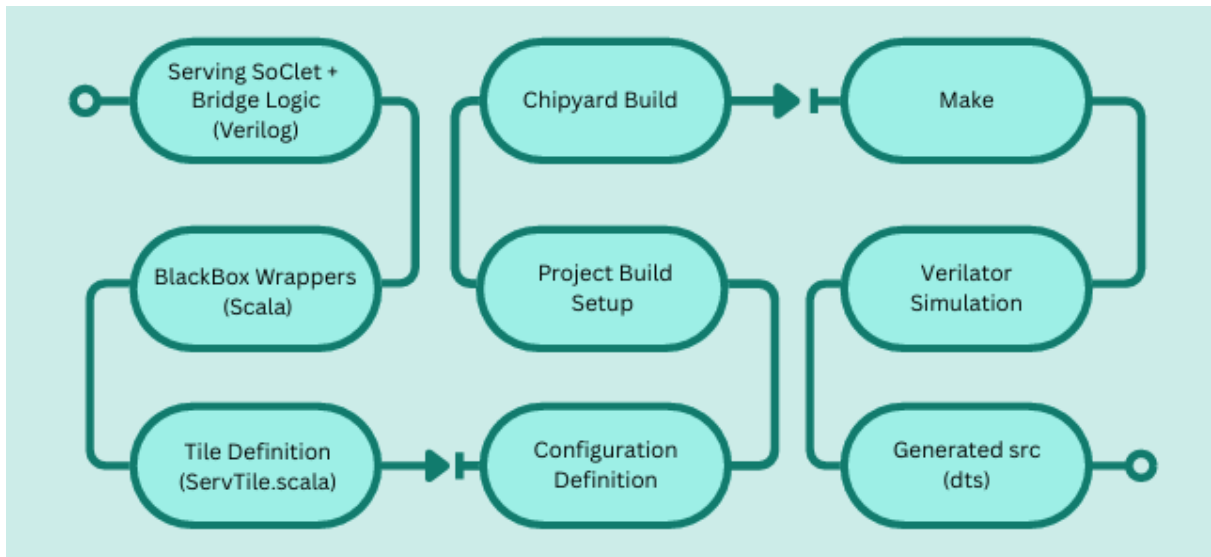


Figure 0-1: Integration Flow

1.2 BlackBox Wrapping for Core Integration

This section discusses how the SERV core plus bridge design was integrated into Chipyard by using a combination of Verilog BlackBox wrappers and Scala BlackBox definitions. The Verilog BlackBox encapsulates the RTL logic, while the Scala BlackBox provides the interface to Chipyard's Chisel/Scala-based flow.

1.2.1 Verilog BlackBox Wrapping

The SERV core along with its Wishbone-to-AXI bridge was first encapsulated inside a **Verilog BlackBox wrapper** (ServCoreBlackbox.v). This step ensures that the complete RTL

subsystem is treated as a single logical module when integrated into Chipyard.

The wrapper serves three main purposes:

1. **Encapsulation of Logic:** The SERV processor and bridge logic are bundled together, so Chipyard only sees one well-defined Verilog module.
2. **Parameter Exposure:** Configurable aspects such as memory size, reset strategy, and CSR support are passed as Verilog parameters (MEMFILE_B, MEMSIZE_B, etc.), making the core flexible.
3. **Standardized Interfaces:** All AXI channels (write, read, response) are explicitly declared, ensuring compatibility with Chipyard's periphery bus through TLToAXI and AXIToTL adapters.

1.2.2 Scala BlackBox Definition

While the Verilog wrapper defines the RTL implementation, Chipyard requires a **Scala BlackBox definition** to instantiate and connect this Verilog module within the Chisel/Scala elaboration flow. The Scala BlackBox is a **Chisel class** that:

1. Mirrors the **parameters** defined in Verilog, using IntParam and StringParam.
2. Defines the **I/O bundle** exactly matching the Verilog ports (clk, rst, i_awaddr, etc.).
3. Links the external Verilog source into the Chipyard build flow using addResource() (if in resources/vsrc) or addPath() (if a preprocessed file is generated).

This ensures that when the Chisel SoC is elaborated, Verilog BlackBox is included transparently and instantiated as if it were a native Chisel module.

The complete file can be found [here](#).

1.2.3 Practical Considerations and Limitations

Although BlackBox wrapping is powerful, it comes with practical considerations:

1. **Interface Matching:** Every signal in Verilog must be mirrored in the Scala IO bundle, both in width and type. A mismatch leads to elaboration or synthesis errors.
2. **Parameter Consistency:** Verilog parameters must have matching Scala parameters. If the Verilog code changes, the Scala wrapper must also be updated.
3. **Debugging Limitation:** Since the BlackBox is opaque to Chisel, internal signals cannot be probed or modified using Chisel utilities (only the ports are visible). Debugging must be done in the Verilog simulation layer.

4. **Build Process Dependencies:** Preprocessing may be required, so the integration flow should be well documented to avoid reproducibility issues.

1.3 Defining Core and Tile Parameters

A new tile wrapper (**ServTile.scala**) must be created to integrate the SERV core into Chipyard. This wrapper specifies the microarchitectural aspects of the SERV core as well as how it is encapsulated in a RocketChip-style tile. This wrapper gives Chipyard the metadata it needs to identify SERV as a legitimate tile type, instantiate it, and use the TileLink and AXI interfaces to link it to the SoC fabric. SERV is a minimal core. Caches, virtual memory, floating-point, atomics, and debugging hardware are therefore not supported. Its core parameters (**ServCoreParams**) and tile parameters (**ServTileParams**) are purposefully kept to a minimum yet adhering to Chipyard's parameterization scheme.

1.3.1 Crafting CoreParams and TileParams case classes

First step is to define **ServCoreParams**, a case class that extends **CoreParams**. This class captures microarchitectural capabilities of SERV, like instruction width, PMP regions, and CSR support. A brief part of **ServCoreParams** is as below. The complete code can be found [here](#).

```
case class ServCoreParams(  
  val bootFreqHz: BigInt = BigInt(320000000),  
  val xLen: Int = 32,  
  val pmpEnable: Int = 0,  
  val pmpGranularity: Int = 0,  
  val pmpNumRegions: Int = 0  
) extends CoreParams { ... }
```

Next, **ServTileParams** extends **InstantiableTileParams** and defines how SERV will exist as a tile in Chipyard. It includes identifiers like `tileId`, tile-level features (`boundaryBuffers`, `clockSinkParams`), and references the **ServCoreParams**. Importantly, both instruction and data caches are explicitly disabled.

```
case class ServTileParams(  
  name: Option[String] = Some("serv_tile"),  
  tileId: Int = 0,  
  val core: ServCoreParams = ServCoreParams()  
) extends InstantiableTileParams[ServTile]
```


1.3.2 InstantiableTileParams

The critical method in `ServTileParams` is **`instantiate()`**, which constructs the actual `ServTile` object during system elaboration. This is where Chipyard translates the high-level configuration into an instantiated tile connected to the SoC interconnect.

The `InstantiableTileParams` part of the `ServTile` is as under.

```
extends InstantiableTileParams[ServTile]
{
  val beuAddr: Option[BigInt] = None
  val blockerCtrlAddr: Option[BigInt] = None
  val btb: Option[BTBParams] = None
  val boundaryBuffers: Boolean = false
  val dcache: Option[DCacheParams] = None //no dcache
  val icache: Option[ICacheParams] = None //no icache -- Bit
serial processing
  val clockSinkParams: ClockSinkParameters =
ClockSinkParameters()
  def instantiate(crossing:
HierarchicalElementCrossingParamsLike, lookup:
LookupByHartIdImpl)(implicit p: Parameters): ServTile = {
    new ServTile(this, crossing, lookup)
  }
}
```

1.3.3 Documentation and handling of “Rocket-specific” parameters

Chipyard’s `TileParams` and `CoreParams` were originally designed around Rocket. Many fields are irrelevant to SERV and are explicitly disabled, while SERV-specific ones were added.

1. Ignored fields:

- `icache`, `dcache`, `btb`, `beuAddr`, `blockerCtrlAddr`: all set to `None`.
- Cache flushing and virtual memory extensions: not supported.

2. Disabled features:

- `useVM` = `false`, `useSupervisor` = `false`, `useDebug` = `false`, `useAtomics` = `false`, `fpu` = `None`.
- No branch predictor, no floating point, no compressed ISA.

3. Added fields:

- MEMSIZE_B, RESET_STRATEGY_B, WITH_CSR_B serve SERV-specific memory model and reset strategy.

Documenting these deviations is essential, as Chipyard tools (e.g., test harness generators, DTS emitters) may otherwise assume Rocket-like defaults.

1.4 The Tile Class

Once the **parameter case classes** are defined, the next step is to create a **Tile class** that extends **BaseTile**. The tile class serves as the integration wrapper that:

1. Instantiates the core inside the RocketChip/Chipyard framework.
2. Connects the tile's **master and slave interfaces** into the SoC fabric using diplomatic TileLink (TL) nodes.
3. Provides device tree and resource bindings to make the tile visible to software.

The **ServTile** is a custom tile that wraps the SERV core. It ensures compatibility with Chipyard's SoC buses and interrupt structures while keeping the minimal feature set of SERV intact.

1.4.1 Custom Class

The ServTile class is defined as a subclass of **BaseTile**. It is parameterized by ServTileParams, which in turn references ServCoreParams. In doing so, SERV is able to communicate with the wider SoC and inherits Chipyard's tile abstraction.

Key features of this class include:

- Inheriting from BaseTile provides standard functionality like clock crossing, interrupt handling, and memory-mapped resource attachment.
- Mixing in SinksExternalInterrupts and SourcesExternalNotifications ensures the core can receive external interrupts and send notifications outward, even though SERV supports only minimal interrupt sources.
- Exposing beatBytes from the Periphery Bus allows the tile to negotiate correct bus widths when interacting with system memory and peripherals.

1.4.2 Connecting TileLink master/slave nodes to diplomatic fabric

Chipyard uses **diplomatic nodes** to abstract communication between tiles and the rest of the SoC. In ServTile, both master and slave interfaces are created and connected:

→ **Master Node (masterNode):**

The tile declares a master interface using the `visibilityNode`, which represents its outbound TileLink connections. This allows the SERV core to act as a **bus master**, issuing memory requests (loads/stores) to the system.

→ **Slave Node**

The tile also has provisions for a slave node (via `tlSlaveXbar.node`), which would expose SERV as a memory-mapped target if required. In the current code, this path is disabled, but the structure allows future expansion.

→ **Xbar Connection:**

The `masterNode` is connected to `tlOtherMastersNode` and then to the master crossbar (`tlMasterXbar`). This ensures the SERV tile's memory transactions are arbitrated alongside other cores or DMA engines in the SoC.

1.4.3 Handling device description and resource registration

Beyond bus connectivity, a tile must also describe itself to the software stack. This is achieved through **device tree entries** and **resource bindings**:

- A `SimpleDevice` descriptor is instantiated inside the `ServTile` to identify the core as a **CPU device**.
- Resource bindings ensure that each tile is uniquely identifiable, with entries such as `hartId` being registered in the device tree.
- These bindings allow Linux or bare-metal runtimes to correctly enumerate and use SERV as a processor in the system.

Although minimal, this step is crucial for making the core visible to higher-level software and ensuring that interrupts, memory maps, and other resources are correctly registered.

```
val cpuDevice: SimpleDevice = new SimpleDevice("cpu",
Seq("OlofKindgren,serv", "riscv")) {
  override def parent = Some(ResourceAnchors.cpus)
  ...
}
ResourceBinding {
  Resource(cpuDevice, "reg").bind(ResourceAddress(tileId))
}
```

1.5 Tile Module Implementation

The `TileModuleImp` serves as the hardware implementation of the previously defined `ServTile` class. While the tile class encapsulates abstract parameters and connections to the SoC fabric, the implementation class instantiates the actual core (in this case, the `ServCoreBlackbox`) and connects it to `TileLink`/`AXI` interfaces. This level bridges **abstract diplomacy nodes** with **concrete Verilog/Chisel I/O wires**, ensuring that the SERV core is functionally integrated into the `RocketChip` subsystem.

1.5.1 TileModuleImp

The `ServTileModuleImp` class extends `BaseTileModuleImp`, which provides common infrastructure such as clock and reset distribution. Inside this class, the SERV core is instantiated as a **Chisel BlackBox** (`ServCoreBlackbox`), ensuring compatibility with the rest of the Chisel-based SoC.

Key steps:

- The SERV Verilog module is wrapped as a Chisel `BlackBox`.
- Parameters are passed to the blackbox from `ServCoreParams`.
- The tile's global clock and reset are connected directly to the core's `clk` and `rst` inputs.

```
class ServTileModuleImp(outer: ServTile) extends
  BaseTileModuleImp(outer) {

  val core = Module(new ServCoreBlackbox (...))
```

1.5.2 Connecting core blackbox to TileLink or AXI interfaces

Once instantiated, the blackbox must be connected to Chipyard's **diplomatic fabric**. This is handled by wiring SERV's AXI signals to the diplomatic nodes (`ServAXI4MNode` for master and `axi4SlaveNode` for slave).

Master Node Connection:

The SERV core acts as an AXI4 **master**, issuing memory requests. Its blackbox outputs (`o_awnvalid`, `o_wmvalid`, etc.) are connected to AXI channels (`aw`, `w`, `ar`), while corresponding

ready/valid signals are fed back to the core. Unused AXI features such as burst, lock, cache, and QoS are tied off or asserted to zero, since SERV implements a minimal interface.

```
outer.ServAXI4MNode.out foreach { case (out, edgeOut) =>

  core.io.i_awmready := out.aw.ready
  out.aw.valid       := core.io.o_awmvalid
  out.aw.bits.addr   := core.io.o_awmaddr ... }
```

Slave Node Connection:

In addition to being a master, SERV also exposes itself as a **slave**, meaning external masters can address it through a memory-mapped region. The slave node connects inbound AXI channels (aw, w, ar) to the core's corresponding blackbox inputs. The SERV core responds with valid/ready signals for the response (b, r) channels. Again, unused fields are set to zero or asserted as not used.

```
outer.axi4SlaveNode.in foreach { case (in, edgeIn) =>
  in.aw.ready       := core.io.o_awready
  core.io.i_awvalid := in.aw.valid
  core.io.i_awaddr  := in.aw.bits.addr
```

1.5.3 Interrupt Handling

Finally, the implementation connects the SERV core's interrupt lines to the broader RocketChip interrupt fabric. Since SERV only supports minimal interrupts, a single timer interrupt is wired through:

```
outer.connectServInterrupts(core.io.i_timer_irq)
```

This ensures that timer events generated in the SoC can be delivered into the SERV core, which is important for correct execution of RISC-V software stacks. Other complex interrupt sources such as software interrupts and external interrupts are not implemented, consistent with SERV's minimalist design.

1.6 Configuring Core in Chipyard Build Flow

After defining the SERV tile and its implementation, the next step is to make the tile selectable in Chipyard's build system. Chipyard relies on a **Config** mechanism, which layers parameter fragments to compose full SoC configurations. This allows adding, removing, or modifying tiles without altering the base code.

The SERV tile is integrated by writing custom **config fragments**, which describe how many instances of the core should be created, how they connect to the subsystem, and what SoC parameters are adjusted accordingly.

1.6.1 Writing Config Fragments to modify TilesLocated.

Chipyard uses the `TilesLocated(InSubsystem)` field to store all tile definitions in the system. To add SERV cores, a new **mixin** class (`WithNServCores`) extends `Config`. This fragment appends one or more `ServTileAttachParams` entries into `TilesLocated`, thereby informing Chipyard that additional SERV tiles exist.

```
class WithNServCores(n: Int = 1) extends Config((site, here, up) => {
  case TilesLocated(InSubsystem) => {
    val prev = up(TilesLocated(InSubsystem), site)
    val idOffset = up(NumTiles)
    (0 until n).map { i =>
      ServTileAttachParams(
        tileParams = ServTileParams(tileId = i + idOffset),
        crossingParams = RocketCrossingParams()
      )
    } ++ prev
  }
  case SystemBusKey => up(SystemBusKey, site).copy(beatBytes = 4)
  case NumTiles => up(NumTiles) + n
})
```

1.6.2 CanAttachTile and TileAttachParams

The `CanAttachTile` trait is implemented by `ServTileAttachParams`, which is necessary for the configuration fragment to function. This guarantees a consistent integration of SERV tiles with the rest of RocketChip's diplomatic framework. Per-core parameters (`name`, `tileId`, `cache`

settings, etc.) are contained in `ServTileParams`. These parameters are bound by `ServTileAttachParams` using a `RocketCrossingParams` structure, which regulates the tile's crossing of the clock and reset domains in relation to the system bus. SERV tiles are attached to the system fabric by utilizing `CanAttachTile`, which eliminates the need for direct subsystem modifications. This modularity is essential because, while elaborating, Chipyard's generator automatically gathers and instantiates every `TilesLocated` entry.

1.7 Protocol Bridging (if Non-TileLink Core)

Instead of using TileLink, many custom cores expose common bus protocols like AXI4 or Wishbone. Adapters are required to integrate any non-TL core into the SoC fabric because the RocketChip generator and Chipyard subsystems are TileLink-native. This guarantees proper communication between the core and the peripherals, memory, and system interconnect.

1.7.1 Identifying interface mismatches

Problem:

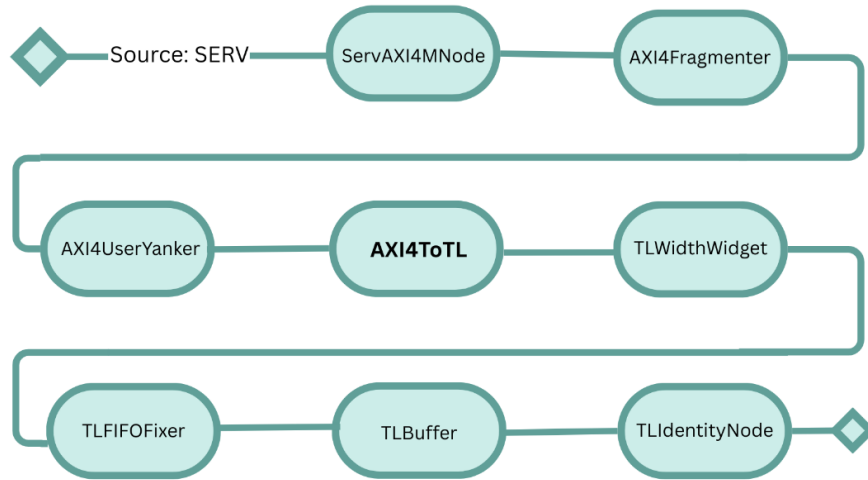
- SERV exposes an **AXI4 master port** for accessing memory.
- It also provides an **AXI4 slave port** for being memory-mapped as a peripheral.
- Chipyard, however, is built around **TileLink diplomacy**, where cores, caches, and buses communicate natively in TL.

Mismatch Examples:

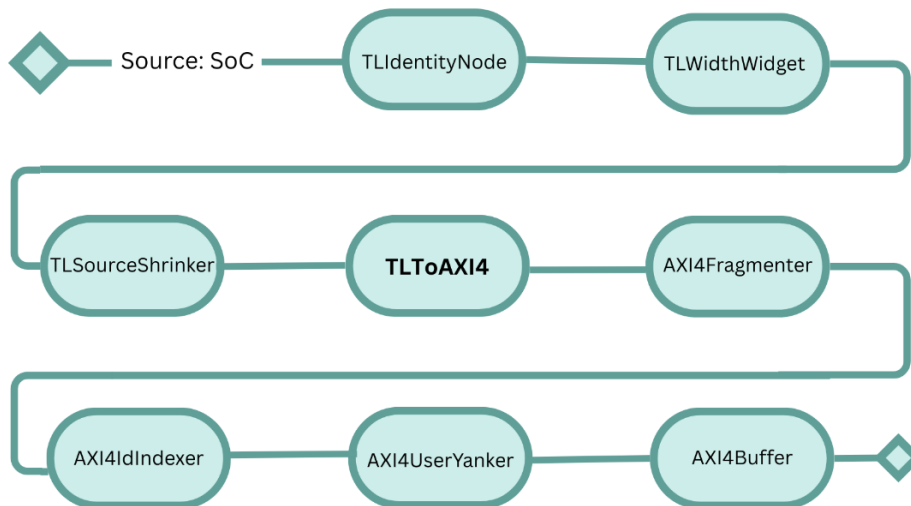
- **AXI4 burst transactions** need to be split into TL fragments.
- **AXI4 user signals** are not meaningful in TL and must be dropped.
- **Width differences** (AXI4 supports varying beat sizes, while TL must be aligned).

1.7.2 Inserting diplomatic adapters

Protocol bridging is handled through a **chain of diplomatic widgets** provided by RocketChip's diplomacy library.

On the Master Side (SERV → System):**Figure 0-2:** AXI-Tilelink Adapter Chain

The SERV core's **AXI4 master interface** generates memory transactions that must be translated into TileLink. The adapter chain first simplifies the AXI protocol (AXI4Fragmenter, AXI4UserYanker), then converts it into TL (AXI4ToTL), and finally aligns bus widths and ordering (TLWidthWidget, TLFIFOFixer, TLBuffer). This allows SERV to act as a fully compatible master on the Chipyard system bus.

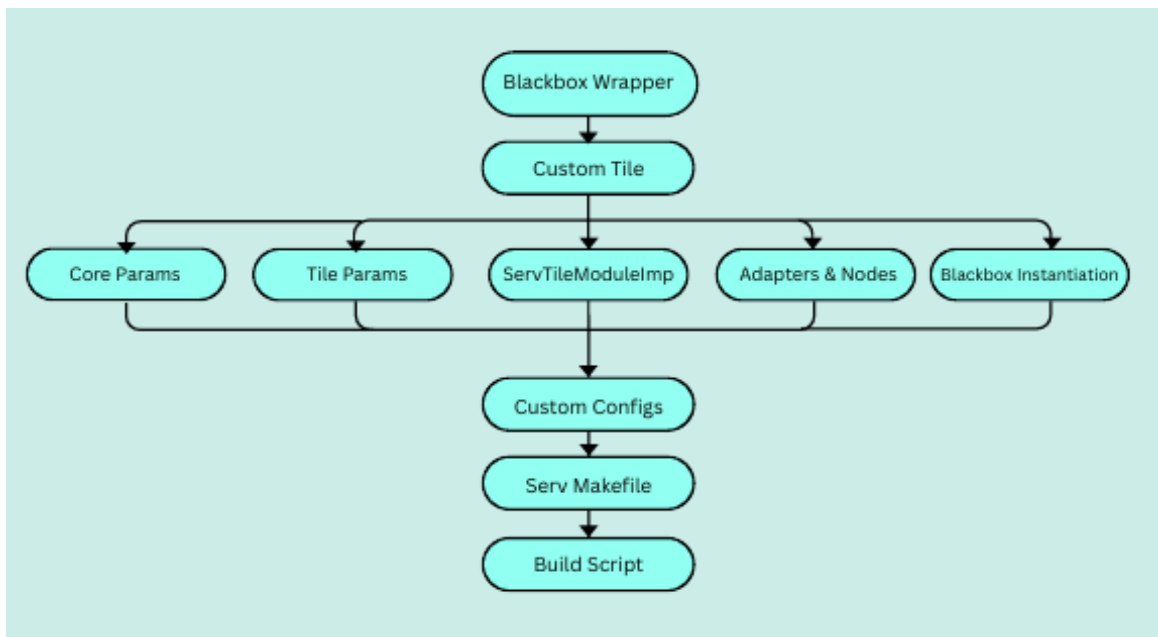
On the Slave Side (System → SERV):**Figure 0-3:** Tilelink-AXI Adapter Chain

The chain slave progressively transforms TL transactions into valid AXI4 operations. This enables the rest of the SoC to access SERV through standard TL while SERV internally continues using AXI4.

1.8 Integration Validation

Integration validation ensures that the custom SERV core is correctly embedded within the Chipyard environment and functions seamlessly alongside native Rocket and BOOM tiles. The process begins by running `make CONFIG=MyCoreConfig` to generate the complete SoC RTL with the SERV tile included. The resulting design is simulated with **Verilator**, where functional tests confirm that the SERV tile communicates properly with the system bus and peripherals.

The **ServTile** wrapper instantiates the SERV core, wires up its clock, reset, and interrupt signals, and exposes it as a TileLink-compatible component. The companion **ServTileModuleImp** class bridges the blackbox core with the diplomatic nodes, ensuring correct translation between AXI4 and TileLink protocols.



I

Figure 0-4: Filesystem for Integration

n practice, SERV primarily uses its **AXI4 master port**, which is converted into TileLink transactions via a chain of adapters (e.g., `AXI4ToTL`, `TLWidthWidget`, `TLFIFOFixer`). While a slave port is also defined, it remains unused in this integration. System-level device entries in the **Device Tree (DTS)** confirm successful recognition of the SERV tile as a first-class CPU device within the SoC. This validation demonstrates that even a minimalist bit-serial RISC-V core like SERV can interoperate within the **Chipyard fabric**, benefiting from the same infrastructure and tooling as more complex cores.

1.9 Summary

The procedure of SERV core integration, the necessary files and configurations required for the reliable integration of a non-Tilelink core with Chipyard, was hereby outlined. Blackboxes, SERV Tile, Configurations and build setup was discussed.

